FloLabs CRM: how Twenty works, how to build on it, and where the analytics tool fits

Based on the code in this repository: the Twenty v2.48.1 fork, branch `automations_branch`, and the CRM Analytics spec v1.0. 5 November 2026.

> **The short version**
>
> - **Twenty is a metadata-driven platform.** Objects and fields are rows in a table, and those rows generate real database tables, APIs and screens. That is why a custom object works with no code.
> - **Build in this order:** configure it, integrate over the API, write a Twenty App, and only then change the fork.
> - **Analytics: your facade idea works, with one change.** Keep the boundary, but make it a module inside the tool rather than a third service. The tool runs in its own container with its own small database, and its screen lives inside Twenty.
> - **Webhooks cannot carry this alone.** In the current code a failed delivery is not retried, anything over 60 calls a minute is dropped, and the payload has no previous stage. Twenty's timeline already records before and after values, so read stage changes from there.
> - **Duplicate storage is not a real cost.** The tool copies IDs and categories, not people, emails or files: roughly 150 MB against a 160 GB disk.

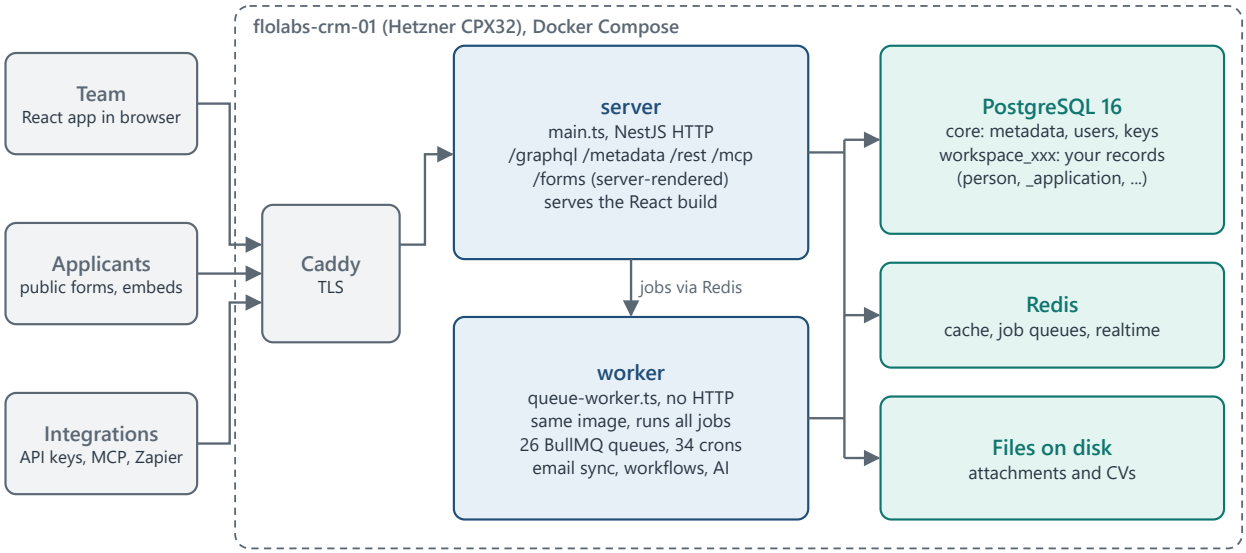
How far this research goes. The repo has about 30,000 TypeScript files, so this is not a line-by-line read. I traced the main paths through each package and read the code behind every claim the design depends on. Nothing was run, and I had no access to the production database. The stage list and the Applications fields live there, so anything marked **verify** is a job for the data audit in Phase 1.

PART 1

How Twenty is built

The system at runtime

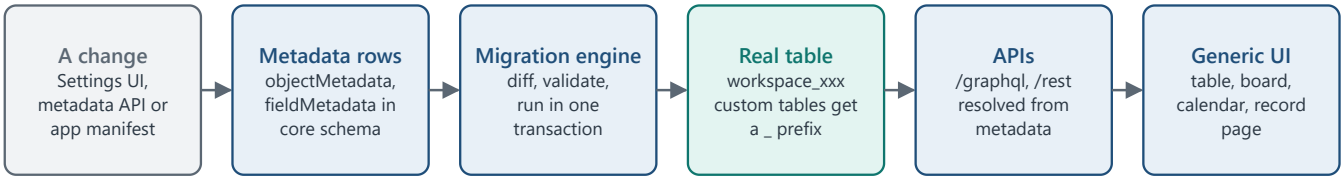
One Docker image runs in two roles. The **server** handles HTTP and only queues background work. The **worker** has no HTTP listener and runs every job. Our fork runs them on one Hetzner box.



Worker also talks to Gmail, Microsoft and IMAP mailboxes, calendars and the Anthropic API. The fork adds no extra containers.

The core idea: metadata drives everything

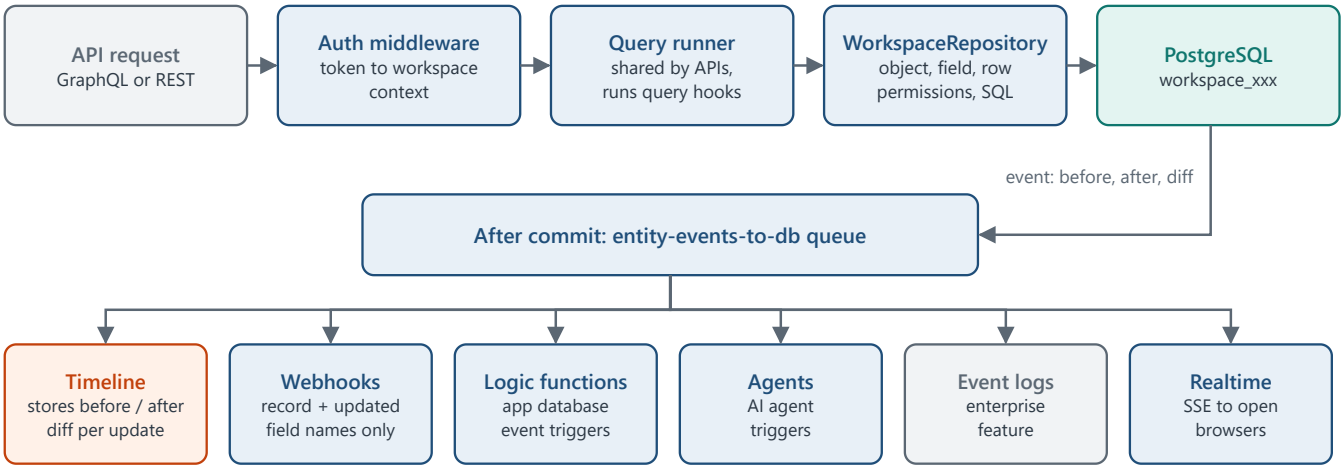
A new field is not a code change. It is a metadata row, and a migration engine turns that row into a real column. The APIs and the generic UI read the same metadata, so they pick it up without a deploy.



- **Everything belongs to an application.** Syncable rows carry a `universalIdentifier` and an `applicationId`. Standard objects like Person are installed by a built-in "standard application" through the same migration engine that apps use. Apps are not bolted on: they use the same path as the core product.
- **Older write-ups are out of date here.** Record queries on `/graphql` skip normal schema execution (`use-direct-execution.hook.ts`). `twenty-orm` now generates its own SQL over `pg`, and TypeORM is left for the `core` schema.

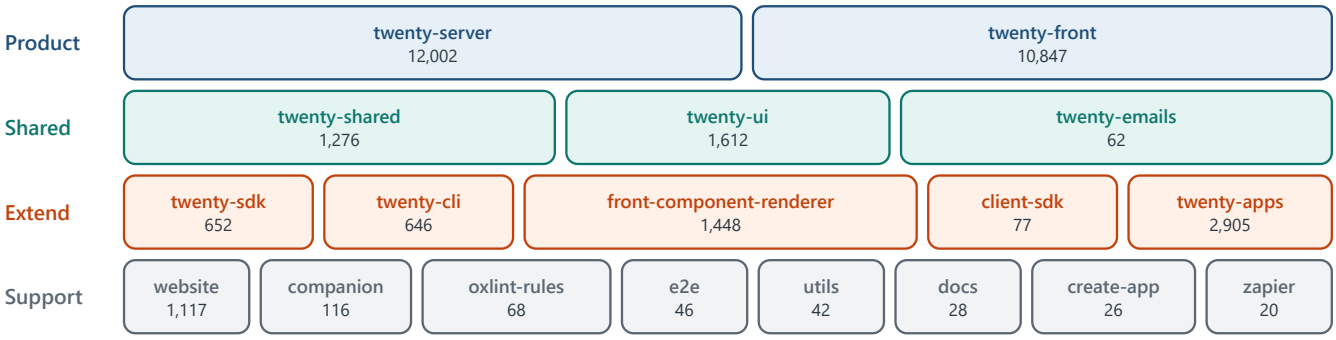
What happens on a write

Every record change takes the same path. Once it commits, one event fans out to all the features that react to data. Any integration is really a choice of where to tap this fan-out.



Orange marks the tap analytics should read from (see Part 2). Workflow triggers run from a separate listener on the same events.

Where the code lives



TypeScript files per package, excluding generated code and `node_modules`.

Backend: twenty-server/src

- `engine/api`: GraphQL, REST, MCP, shared query runners.
- `engine/metadata-modules`: objects, fields, roles, views, webhooks, AI (89 folders, 35 of them `flat-*` caches).
- `engine/core-modules`: auth, files, queues, billing, tools (about 85 folders).
- `engine/twenty-orm`, `workspace-manager`: data access and migrations.
- `modules/`: the product, 25 folders such as messaging, calendar, workflow, timeline. Ours are `public-form` and `notifications-seen`.

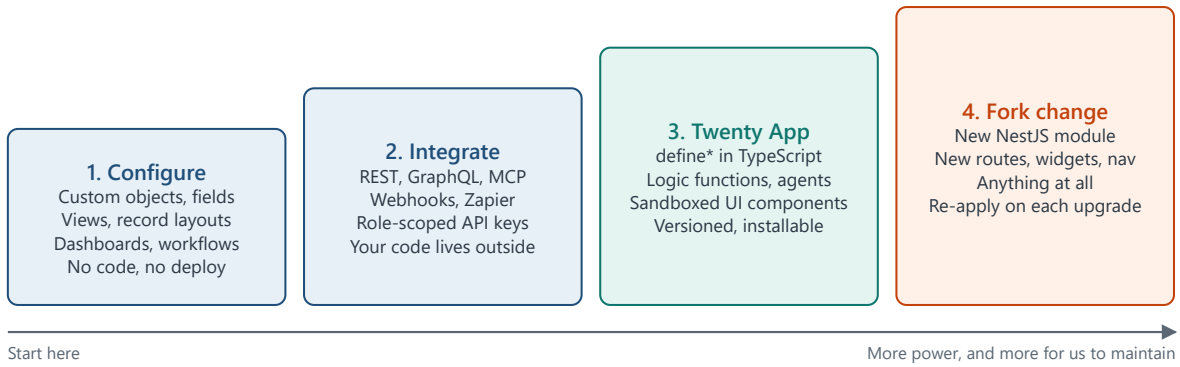
Frontend: twenty-front/src/modules

- **Boot:** load client config, restore metadata from IndexedDB, refetch whatever is stale.
- **State:** Jotai, with component state keyed by instance.
- **Data:** three Apollo clients for `/metadata`, `/graphql` and `/admin-panel`.
- `object-record`: builds queries from metadata and renders any object.
- `page-layout`: tabs and widgets on record pages and dashboards.
- **Ours:** `form-builder`, `team-inbox`, `notifications`.

Also built in. A workflow engine with 4 trigger types and 22 action types, including HTTP, code, AI agent and email. AI agents, chat and skills, with 11 tool providers exposed to agents and an MCP endpoint at `POST /mcp`. Email and calendar sync across Gmail, Microsoft, IMAP and CalDAV.

How to build on top of it

Pick the lowest step that does the job. Each step up gives more power and costs more to keep alive through upgrades.



Step	Use it for	What to watch
1. Configure	New objects, fields and views. Native dashboards with aggregate, pie, bar and line charts.	Workflows stay off until the team turns them on (FLOLABS.md).
2. Integrate	External tools and services that keep their own runtime. This is where the analytics backend sits.	Default API limit is 100 requests a minute, with 200 records a page. The webhook docs show the wrong payload shape, so trust the code.
3. Twenty App	Features that should live inside the CRM and be versioned: `defineObject`, `defineLogicFunction`, `defineFrontComponent`, `defineAgent`, `defineSkill`, `defineRole`, `definePageLayout`, `defineNavigationMenuItem`.	See the two limits below. No FloLabs app exists yet.
4. Fork change	Things apps cannot reach: nav, branding, email behaviour, a new native screen. Forms, inbox and notifications all live here.	Every upgrade replaces the tree and cherry-picks our commits, so each file we touch can conflict. Upgrade commands have strict rules (`UPGRADE_COMMANDS.md`).

What an app looks like

```
export default defineLogicFunction({
  universalIdentifier: '...',
  name: 'on-opportunity-created',
  handler,
  databaseEventTriggerSettings: {
    eventName: 'opportunity.created',
    batchMode: true,
  },
});
```

Trimmed from `twenty-apps/public/last-contact`. Other triggers: `cronTriggerSettings`, `httpRouteTriggerSettings`. Good examples to copy: `examples/hello-world`, `public/fathom`, `public/slack`.

```
yarn twenty remote:add --url https://crm...
yarn twenty app:publish --private
yarn twenty app:install
```

Two limits on our single server

- **Logic functions are off by default in production.** `LOGIC_FUNCTION_TYPE` defaults to `DISABLED`, which also breaks workflow Code actions. Without AWS Lambda the only option is `LOCAL`. That runs a child process that inherits the server's environment and is not sandboxed, so app code can read our secrets. Only run code we trust. The production setting is not in the repo: **verify**.
- **Front components run in a sandbox with a partial DOM.** There is no `ResizeObserver`, so responsive chart libraries are a risky fit.

Licence

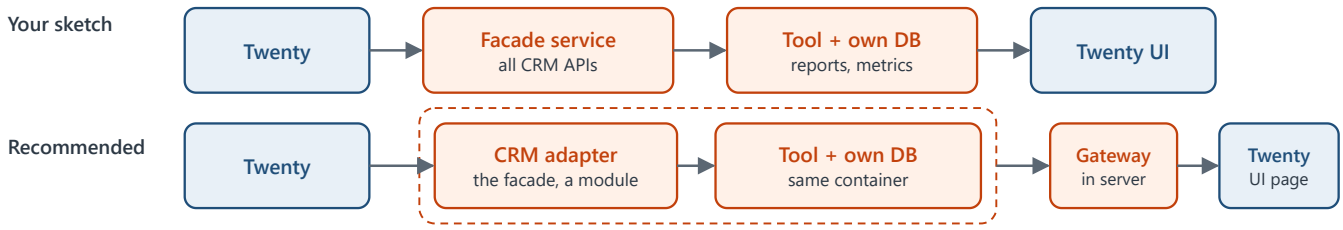
AGPLv3 means we must offer our modified source to users. Enterprise-marked code covers SSO, row-level permissions, record sharing, event logs and usage limits, and stays off without a subscription. App, webhook, API key, workflow and MCP code carry no such marker.

PART 2

Architecture for the CRM Analytics tool

Your facade idea

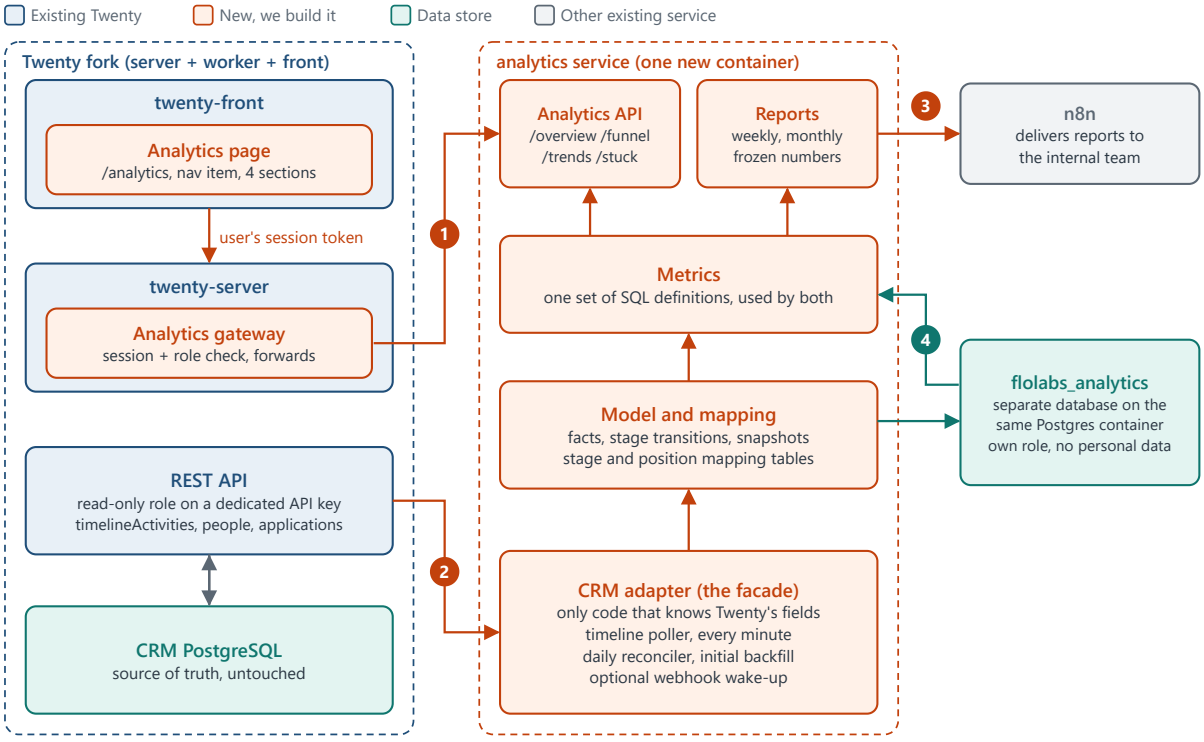
> **The boundary is right. A separate facade service is more than we need today.** The value of a facade is that only one place knows Twenty's field names and quirks. A module gives us that. A third container only adds another thing to deploy and monitor, with one consumer behind it. If a second tool needs the same data, a clean module is easy to lift out.



"Facade" also covers two different jobs, and they belong in different places:

- **Data coming in: the CRM adapter,** inside the tool. It handles polling, backfill and reconciliation, and it translates Twenty's records into the tool's own terms.
- **Requests from the screen: the gateway,** a thin module inside `twenty-server`. It checks the user's session and role, then forwards to the tool. This means no second login, and the tool never has to be exposed publicly.

Proposed architecture



1	The page calls the gateway with the normal user session. The gateway calls the Analytics API over the private Docker network with a service token.
2	The adapter reads through Twenty's REST API with a dedicated API key whose role can only read. Twenty enforces that, so the read-only rule holds even if the tool has a bug.
3	Reports freeze their numbers when generated. n8n, already running on our other server, delivers them to the internal team only.
4	A separate database in the same Postgres container, with its own role. The existing `pg_dumpall` in `deploy.sh` and the Hetzner backups cover it with no extra work.

What the code changes about the spec

The spec asks to verify event coverage and historical availability (section 10). Reading the code answers most of that, and it changes the plan.

Spec assumes	The code does	Source
Retries recover failed webhooks	`retryLimit: 3` is set, but `callWebhook` catches every error, so the job never fails and never retries. Timeout is 5 seconds. If the tool is down, those events are gone.	call-webhook.job.ts:141
Webhooks report every change	Default limit is 60 calls a minute per workspace, and the excess is dropped with a log line. A bulk stage change on 500 people would lose about 440 events. The limit is configurable.	webhook-rate-limit.service.ts
Events describe the transition	The payload holds the new record and the changed field names. There is no previous value.	transform-event-to-webhook-event.ts
Tool can receive on the same box	Webhook URLs resolving to private IPs are blocked unless listed in `OUTBOUND_HTTP_ALLOWED_INTERNAL_HOSTS`.	secure-http-client.service.ts
History may be unavailable	**History exists from go-live.** Timeline rows keep `diff: {before, after}` for audit-logged fields, with a timestamp. Changes from before the HubSpot move are not there unless they were imported: **verify**.	timeline-activity.service.ts
Everything keys on application ID	FLOLABS.md puts `Stage` on **Person**, so someone with two applications likely has one stage. Applications holds 42,371 records from HubSpot, and its fields are not in the repo: **verify**.	FLOLABS.md section 2
Track and program exist	Not as CRM fields in the repo. `positionsAppliedFor` is a flat multi-select with 39 values. Five tracks appear as a question in two draft forms, but those answers map to no person field.	forms/skills-experience.json

How the tool should get its data

Approach	Previous stage?	If the tool is down	Bulk changes	Twenty change	Verdict
Webhooks + daily reconcile (spec)	No	Events lost. Reconcile fixes the state but not the times.	Over 60 a minute dropped	Config only	Not on its own
Poll the timeline over the API	Yes	Resumes from its cursor, nothing lost	Safe	None	**Recommended**
Read Twenty's Postgres directly	Yes	Safe	Safe	New DB role	Ties us to internal tables, bypasses permissions
Small listener in twenty-server	Yes, every change	Safe with a real retrying queue	Safe	Fork code	Fallback if the audit fails

How it works. Each minute, the adapter fetches timeline rows for person updates, starting 15 minutes before its last position. It upserts them by timeline row ID, so re-reading is harmless and the spec's idempotency rule is met. The cursor does the job of the spec's retry queue, so that queue goes away. One minute is fresh enough for a dashboard, and a webhook can be added later purely as a "check now" signal. On the spec's Authentication and Validation step: polling leaves nothing inbound to authenticate, and an added webhook would check its HMAC signature. Field and timestamp validation stays in the adapter. The daily reconcile stays: it compares current stage against the CRM and records any gap as an estimated transition, flagged as such.

One limit to design around. Twenty merges timeline updates from the same person on the same record within 10 minutes (`timeline-activity.repository.ts`). A to B to C becomes A to C, and the row changes in place, which is why we re-read the last 15 minutes. Quick corrections collapsing is usually what we want. It does mean the funnel must count reaching a later stage as having passed the earlier ones, which people skipping stages by hand requires anyway.

First test in the audit: with a read-only key, confirm `timelineActivities` is readable over the API and that `stage` changes show up in the diff. If it fails, use the listener instead. The rest of the design stays the same.

Your two notes on the spec

"Why normalize?"

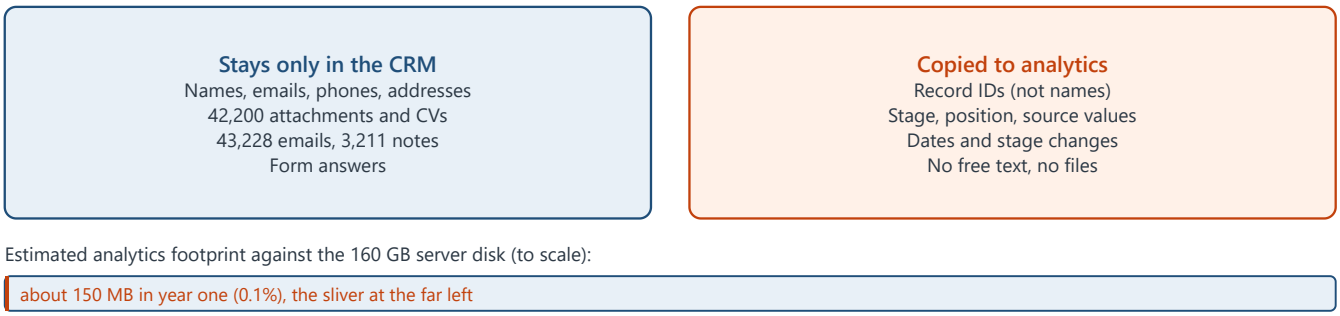
Because the CRM's raw values do not answer the questions the dashboard asks. There are three concrete problems:

- **Stages mix two things.** The spec wants lifecycle stages kept apart from reminder substages. A flat list of 25 options cannot mark which is which, and we need a funnel order we control.
- **Programs have no grouping and no single spelling.** The CRM has 39 flat positions, while the forms name 5 tracks such as "Robotics & Intelligent Systems". One program appears as `SPORT_SCIENCE_INTERN`, "Sport Science" and "Sports Science", depending on the source. Without a mapping, a track view is not possible and one program can count as three.
- **Labels change.** If someone edits a stage in Settings, history must not split into two stages.

> **So keep it, but make it small:** two lookup tables, not a pipeline step. Store the raw value in the history, and join to the mapping when querying. If a definition changes, we edit one row and all of history follows, with nothing to rewrite. A raw value with no mapping shows up as "Unmapped" and raises an alert.

"Won't another database duplicate the data?"

Your concern is fair, and the answer is to copy very little. We can adopt your suggestion for most of the data, but not all of it.



Estimate, assuming 5 stage changes per person and about 1,500 snapshot rows a day. Measure this in Phase 1.

Dataset	Retention	If lost	Why we keep it
Raw fetched events	30 days, then deleted	Nothing, re-read	Debugging only
Current state	Overwritten in place	Rebuild from CRM	Filters, current counts, reconcile comparison
Stage transitions	**Forever**	**Hard to replace**	Conversion, drop-off and duration all come from this
Daily snapshots	Forever, or roll up to monthly later	Partly rebuildable	Trends and consistency checks
Report results	Forever, never recalculated	Regenerate, may differ	What the team saw at the time

Why results alone are not enough. A March cohort's conversion keeps changing as those people progress, so it has to be recalculated from per-person history. When someone redefines "converted" or asks for a new filter, stored totals cannot answer, while stored transitions can. And the spec itself says not to manufacture history, so a change we did not record is gone for good.

Building the tool

A single modular service, as in the diagram: 43,000 people is a small dataset for Postgres, so it does not need a warehouse or stream processing.

Layer	Recommendation	Why
Backend	TypeScript with NestJS	Same language and patterns as the fork. FastAPI is fine if the builder prefers Python, since the metrics are SQL either way.
Database	Separate `flolabs_analytics` database in the existing Postgres 16 container	Backups already covered. Add a statement timeout, and move it to its own container if it ever competes with the CRM.
Metrics	SQL views and functions in one folder, materialized only if slow	Deterministic, and dashboard and reports cannot disagree. No ClickHouse or dbt.
Jobs	In-process scheduler for poll, reconcile and snapshot; no queue	The cursor already covers retries. `sync_log` records every run.
Screen	Module in `twenty-front`, not Next.js	You want it inside Twenty, and `twenty-ui` and chart code are already here. A funnel chart must be built, since none exists.
Monitoring	Last sync time on every screen; alert on stale data	Compare totals with `GET /rest/people/groupBy` after each reconcile.

Data model

```
person_fact      person_id PK, stage_raw, stage_entered_at, source_raw, created_at, deleted_at
person_position  person_id, position_raw                    -- one person can have several
application_fact application_id PK, person_id, applied_at    -- fields to verify
stage_transition id, person_id, from_raw, to_raw, occurred_at, origin (timeline|reconcile|backfill),
                 source_ref UNIQUE, is_estimated
dim_stage        raw PK, canonical, funnel_order, kind (lifecycle|reminder|terminal)
dim_position     raw PK, program, track
daily_snapshot   date, stage, track, program, source, people, applications
report_run       id, kind, period, metrics JSONB, definition_version, generated_at
sync_log         id, job, started_at, finished_at, status, records, errors
```

Counting rules to fix early. A person with three positions appears in three tracks, so track totals add up to more than total people: compute the total separately, never by summing. Report conversion by application-date cohort, as the spec says. Drop-off counts someone only after the observation window ends.

How it plugs into Twenty

Option	How	Good	Bad
A. Native page in the fork (recommended)	New route, nav item and front module, plus a gateway module in the server	Looks native, no second login, role-based access, same pattern as Team inbox	Adds fork code to cherry-pick; UI changes need a full image build (about 15 min)
B. Iframe widget	Twenty's `IFRAME` widget on a dashboard, pointing at a separate web app	No fork change, deploys on its own	Passes no user identity, so it needs its own login, and it feels embedded
C. Twenty App	`defineFrontComponent` + `defineNavigationMenuItem`	No fork change, versioned	Sandbox has a partial DOM, so charts are risky. Calling our API from it is untested.
D. Native dashboards only	Chart widgets on CRM data, no tool	Works today with no code	Current counts only: no funnel, no history, no cohorts

Do D now as a quick win: a stage distribution dashboard gives the team something in an afternoon and covers part of Pipeline Overview. **Then build A**. Later, a stage history widget could go on the person page.

What changes in the repo for option A

Change	Where
Route and path	`twenty-shared/src/types/AppPath.ts`, `app/routing/utils/createWorkspaceRouteObjects.tsx`
Sidebar item	`navigation/components/MainNavigationDrawerScrollableItems.tsx`
Screens and charts	`twenty-front/src/modules/analytics` (new); person links use `/object/person/:id`
Gateway	`twenty-server/src/modules/analytics-gateway` (new), imported in `modules.module.ts`; guarded like the REST controller with `JwtAuthGuard` and a role check
Read-only access	New role and API key in Settings, no code
Deployment	New `analytics` service in `/opt/flolabs-crm` Compose, on the private network with no Caddy route; its own image and deploy, since `deploy.sh` only touches server and worker

Build plan

This follows the spec's phases. Durations are rough, for one developer.

Phase	Deliverables	Done when
1. Data audit 2 to 3 days	Run the timeline test with a real read-only key. Export the 25 stages and Applications fields. Check whether HubSpot history can be backfilled. Add the quick-win dashboard.	Decisions below are settled and the mapping tables are filled
2. Pipeline about 1 week	Service skeleton, adapter, backfill, poller, reconcile, mapping, snapshots	Totals match the CRM and replaying events changes nothing
3. Metrics and screen about 2 weeks	SQL metrics, Analytics API, gateway, the four dashboard sections and filters	Metrics match a hand-calculated sample
4. Reports and monitoring about 1 week	Weekly and monthly reports, n8n delivery, stale-data alerts, retention jobs	Reports run on schedule, and a failed job is detected and recoverable

What we need to decide together

#	Question	My default
1	**Facade:** a module in the tool, or its own service?	Module. Split it out when a second consumer appears.
2	**Funnel unit:** person or application? Stage is on Person today.	Person for the funnel, applications for volume counts. Moving stage to Application would be a separate CRM change.
3	**UI:** are we fine adding a native page to the fork?	Yes (option A). It matches Team inbox and Forms.
4	**Mapping:** who decides which of the 25 stages are lifecycle or reminder, and how the 39 positions group into programs and tracks?	Recruiting owns it, starting from the 5 tracks already in the forms. We need this before Phase 2 ends.
5	**Success:** which stage counts as final, such as enrolled or onboarded?	Needs the recruiting team.
6	**Rules:** reporting timezone, drop-off window, stuck thresholds?	Store UTC. The rest are config values, with defaults set by recruiting.
7	**History:** backfill stage history from HubSpot?	Try it if HubSpot still exposes it (its API has a property history option; not tested). Otherwise start from go-live and label trends clearly.
8	**Access:** which roles see analytics, and who gets reports?	A new permission on selected roles. Reports go to a fixed internal list.
9	**Build or buy the screens?** Metabase, which your spec references, could sit on the same analytics database.	Build, because you want it inside Twenty. The pipeline does not depend on this, so Metabase stays open. I have not evaluated Metabase here; it would mean a separate login and more memory on this server.